

SIKSHA 'O' ANUSANDHAN
(Deemed To Be University)
Accredited (3rd Cycle) by NAAC with A⁺⁺ Grade

Admission Batch: 2023

Session: 2025-26

**Major Project Report of
MACHINE LEARNING CONCEPT -2 (CSE 3968)
On**

Handwritten Digit Recognition Using Deep Learning

Submitted by:

Name	Registration number
1) Debadrita Konar	2341011084
2) Utkarsh Singh	2341019539
3) Sarthak Raj	2341018043
4) Archita Panigrahy	2341016162
5) Suryansh Mohanty	2341013313

Section: 23412E2

Semester: 6th

Branch: CSE



**Department of Computer Science & Engineering
Institute of Technical Education and Research (ITER), S'O'A
Jagamohan Nagar, Jagamara, Bhubaneswar, Odisha – 751030**

May, 2025

1. Abstract

This project presents an automated Handwritten Digit Recognition system developed using classical machine learning models trained on the MNIST dataset. Three classification algorithms were implemented and evaluated: Multi-Layer Perceptron (MLP) Neural Network, Support Vector Machine (SVM) with an RBF kernel, and K-Nearest Neighbors (KNN). The MNIST dataset comprises 70,000 grayscale images of handwritten digits (0-9), each of size 28x28 pixels. Images were preprocessed through flattening and normalization before being fed into the classifiers. Comparative results demonstrate that the MLP model achieved the highest accuracy of 93.25%, followed by SVM at 93.10% and KNN at 89.15%. The project demonstrates that deep learning approaches outperform traditional classifiers for image recognition tasks, and provides a foundation for future improvements using Convolutional Neural Networks (CNNs).

2. Table of Contents

1. Abstract	2
2. Table of Contents	2
3. Introduction	3
4. Problem Statement	3
5. Objectives	3
6. Literature Review	4
7. Methodology	4
8. System Architecture	6
9. Implementation	6
10. Results	9
11. Discussion	10
12. Applications	10
13. Future Scope	11
14. Conclusion	11
15. References	11
16. Individual Contributions Table	12

3. Introduction

Handwriting recognition is a classic problem in the domain of computer vision and pattern recognition. Despite significant advances, recognizing handwritten characters reliably remains challenging due to wide variability in writing styles, stroke thickness, slant, and noise. Automated handwritten digit recognition has broad real-world applications including postal mail sorting, bank cheque processing, digitization of handwritten forms, and accessibility tools for persons with disabilities.

Machine learning, particularly deep learning, has revolutionized how such visual tasks are approached. Rather than relying on hand-crafted feature extraction, neural networks learn hierarchical representations directly from raw pixel data. This project investigates three algorithmic approaches on the benchmark MNIST dataset: a Multi-Layer Perceptron (MLP) neural network, a Support Vector Machine (SVM), and the K-Nearest Neighbors (KNN) algorithm. Each method represents a different paradigm in machine learning, and comparing them provides valuable insight into the trade-offs between accuracy, training time, and computational cost.

4. Problem Statement

Manual digit recognition processes are inherently error-prone, time-consuming, and not scalable. In contexts like postal services or banking, human operators must visually inspect and interpret handwritten numerals, which introduces fatigue-related inaccuracies and bottlenecks. This project aims to automate the recognition of handwritten digits (0-9) using supervised machine learning techniques, trained on the well-known MNIST benchmark dataset, in order to achieve high-accuracy, real-time digit classification without human intervention.

5. Objectives

The primary objectives of this project are:

- To implement and compare MLP, SVM, and KNN classifiers for digit recognition
- To preprocess the MNIST dataset through normalization and feature flattening
- To evaluate each model using accuracy, classification reports, and confusion matrices
- To visualize model performance through comparison graphs and training loss curves
- To provide a framework extensible to future deep learning improvements such as CNNs

6. Literature Review

The MNIST dataset, introduced by LeCun et al. (1998), has been a de facto standard benchmark for evaluating handwritten digit recognition systems. Early work employed classical statistical methods such as linear discriminant analysis and nearest-neighbor classifiers. Lecun et al. demonstrated that Convolutional Neural Networks (LeNet-5) could achieve error rates below 1% on MNIST, establishing the superiority of deep learning approaches for image tasks.

Support Vector Machines with RBF kernels have been extensively applied to digit classification, with Scholkopf et al. and others documenting accuracy above 98% on MNIST using optimal hyperparameter tuning. However, SVM scalability becomes a concern for large datasets. Multi-Layer Perceptrons trained with backpropagation and modern optimizers (Adam, RMSProp) achieve strong performance with relatively short training times. KNN is often employed as a baseline classifier; its simplicity and absence of a training phase make it attractive, though high memory and inference costs limit deployment. This project implements all three approaches to compare their practical performance characteristics on a representative subset of MNIST.

7. Methodology

7.1 Dataset

The MNIST (Modified National Institute of Standards and Technology) dataset is loaded directly via TensorFlow/Keras. It contains 60,000 training images and 10,000 test images, each a 28x28 grayscale image of a handwritten digit from 0 to 9. The dataset is well-balanced across all ten classes, with approximately 5,800 to 6,700 training samples per class.

7.2 Data Preprocessing

The 28x28 images are flattened into 784-dimensional feature vectors. Pixel values originally in the range [0, 255] are normalized to [0, 1] by dividing by 255.0. This normalization ensures numerical stability and faster convergence during training. For computational efficiency, a subset of 10,000 training samples and 2,000 test samples is used for SVM and KNN. Features are further standardized using StandardScaler (zero mean, unit variance) prior to MLP and SVM training.

Code Snippet - Step 1: Import Libraries

```
import numpy as np import pandas as pd import matplotlib.pyplot as plt import seaborn
as sns import warnings, time warnings.filterwarnings('ignore') from
tensorflow.keras.datasets import mnist from sklearn.neural_network import
MLPClassifier from sklearn.svm import SVC from sklearn.neighbors import
KNeighborsClassifier from sklearn.metrics import accuracy_score,
classification_report, confusion_matrix from sklearn.preprocessing import
StandardScaler
```

This block imports all required libraries. NumPy and Pandas handle numerical operations and data structures. Matplotlib and Seaborn provide visualization utilities. The MNIST dataset is fetched from TensorFlow's Keras datasets API. Scikit-learn provides the three classifiers (MLPClassifier, SVC, KNeighborsClassifier) along with evaluation and preprocessing utilities.

Code Snippet - Step 2: Load Dataset and Preprocessing

```
(X_train_raw, y_train), (X_test_raw, y_test) = mnist.load_data() # Flatten 28x28
images to 784-dimensional vectors X_train_flat =
X_train_raw.reshape(X_train_raw.shape[0], -1).astype('float32') X_test_flat =
X_test_raw.reshape(X_test_raw.shape[0], -1).astype('float32') # Normalize pixel
values [0, 255] to [0, 1] X_train_norm = X_train_flat / 255.0 X_test_norm =
X_test_flat / 255.0 # Subset for SVM and KNN TRAIN_SUBSET = 10000; TEST_SUBSET =
2000 X_tr = X_train_norm[:TRAIN_SUBSET]; y_tr = y_train[:TRAIN_SUBSET] X_te =
X_test_norm[:TEST_SUBSET]; y_te = y_test[:TEST_SUBSET] # Standardize (zero mean,
unit variance) scaler = StandardScaler() X_tr_scaled = scaler.fit_transform(X_tr)
X_te_scaled = scaler.transform(X_te)
```

The dataset is loaded and split into training (60,000) and test (10,000) sets. The reshape operation transforms each 28x28 matrix into a flat vector of 784 elements. Division by 255.0 maps pixel intensities to the range [0, 1], preventing any single feature from dominating due to magnitude differences. StandardScaler subtracts the mean and divides by the standard deviation, which is essential for SVM and MLP convergence. A 10,000-sample training subset is used for computational tractability with SVM and KNN.

8. System Architecture

The system pipeline is structured as follows:

- Input Layer: Raw 28x28 grayscale digit images from the MNIST dataset
- Preprocessing: Flattening to 784-D vectors, normalization, StandardScaler standardization
- Model Training: Three parallel classifiers trained independently (MLP, SVM, KNN)
- Prediction: Each model outputs a digit class label (0-9) for test inputs
- Evaluation: Accuracy scores, confusion matrices, and comparison graphs are generated

9. Implementation

Tools & Technologies:

- Python
- Libraries: NumPy,Pandas,Sickit-Learn,TensorFlow
- Jupyter Notebook

Steps:

▼ Step 1: Install & Import Libraries

```
[1]: # Install required libraries (run once if not already installed)
# !pip install scikit-learn tensorflow matplotlib seaborn numpy pandas

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
import warnings
import time
warnings.filterwarnings('ignore')

# Dataset
from tensorflow.keras.datasets import mnist

# Models
from sklearn.neural_network import MLPClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier

# Evaluation
from sklearn.metrics import (
    accuracy_score, classification_report,
    confusion_matrix, ConfusionMatrixDisplay
)
from sklearn.preprocessing import StandardScaler

print(" All libraries imported successfully!")

All libraries imported successfully!
```

Step 2: Load the MNIST Dataset

```
[3]: # Load MNIST dataset
(X_train_raw, y_train), (X_test_raw, y_test) = mnist.load_data()

print(f"Training data shape : {X_train_raw.shape} → {X_train_raw.shape[0]} images of {X_train_raw.shape[1]}x{X_train_raw.shape[2]} pixels")
print(f"Test data shape : {X_test_raw.shape} → {X_test_raw.shape[0]} images of {X_test_raw.shape[1]}x{X_test_raw.shape[2]} pixels")
print(f"Unique classes : {np.unique(y_train)}")

Training data shape : (60000, 28, 28) → 60000 images of 28x28 pixels
Test data shape : (10000, 28, 28) → 10000 images of 28x28 pixels
Unique classes : [0 1 2 3 4 5 6 7 8 9]
```

Step 3: Visualize Sample Images

```
[4]: fig, axes = plt.subplots(3, 10, figsize=(18, 6))
fig.suptitle('■ Sample MNIST Images (Digits 0-9)', fontsize=16, fontweight='bold', y=1.02)

for digit in range(10):
    indices = np.where(y_train == digit)[0][:3]
    for row, idx in enumerate(indices):
        axes[row, digit].imshow(X_train_raw[idx], cmap='gray')
        axes[row, digit].axis('off')
        if row == 0:
            axes[row, digit].set_title(f'Digit {digit}', fontsize=11, fontweight='bold')

plt.tight_layout()
plt.savefig('sample_images.png', dpi=120, bbox_inches='tight')
plt.show()
print("Sample images visualized!")
```

Step 4: Explore Class Distribution

```
[5]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Class Distribution in MNIST Dataset', fontsize=14, fontweight='bold')

colors = plt.cm.tab10(np.linspace(0, 1, 10))

# Training distribution
train_counts = np.bincount(y_train)
axes[0].bar(range(10), train_counts, color=colors, edgecolor='black', linewidth=0.7)
axes[0].set_title('Training Set (60,000 samples)', fontsize=12)
axes[0].set_xlabel('Digit Class')
axes[0].set_ylabel('Count')
axes[0].set_xticks(range(10))
for i, count in enumerate(train_counts):
    axes[0].text(i, count + 50, str(count), ha='center', va='bottom', fontsize=9)

# Test distribution
test_counts = np.bincount(y_test)
axes[1].bar(range(10), test_counts, color=colors, edgecolor='black', linewidth=0.7)
axes[1].set_title('Test Set (10,000 samples)', fontsize=12)
axes[1].set_xlabel('Digit Class')
axes[1].set_ylabel('Count')
axes[1].set_xticks(range(10))
for i, count in enumerate(test_counts):
    axes[1].text(i, count + 10, str(count), ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.savefig('class_distribution.png', dpi=120, bbox_inches='tight')
plt.show()
```

Step 5: Preprocessing

```
[6]: # Flatten 28x28 images → 784-dimensional vectors
X_train_flat = X_train_raw.reshape(X_train_raw.shape[0], -1).astype('float32')
X_test_flat = X_test_raw.reshape(X_test_raw.shape[0], -1).astype('float32')

# Normalize pixel values from [0, 255] → [0, 1]
X_train_norm = X_train_flat / 255.0
X_test_norm = X_test_flat / 255.0

# Use a subset for SVM and KNN (faster training, still representative)
TRAIN_SUBSET = 10000
TEST_SUBSET = 2000

X_tr = X_train_norm[:TRAIN_SUBSET]
y_tr = y_train[:TRAIN_SUBSET]
X_te = X_test_norm[:TEST_SUBSET]
y_te = y_test[:TEST_SUBSET]

# Scale for SVM and MLP (zero mean, unit variance)
scaler = StandardScaler()
X_tr_scaled = scaler.fit_transform(X_tr)
X_te_scaled = scaler.transform(X_te)

print(f"Training subset : {X_tr.shape[0]} samples")
print(f"Test subset      : {X_te.shape[0]} samples")
print(f"Feature vector   : {X_tr.shape[1]} dimensions (28x28)")
print("✅ Preprocessing complete!")

Training subset : 10000 samples
Test subset      : 2000 samples
Feature vector   : 784 dimensions (28x28)
✅ Preprocessing complete!
```

Step 6: Train Models

6.1 — Multi-Layer Perceptron (MLP)

```
[10]: print("Training MLP Neural Network...")
start = time.time()

mlp = MLPClassifier(
    hidden_layer_sizes=(256, 128, 64), # 3 hidden layers
    activation='relu',
    solver='adam',
    max_iter=30,
    random_state=42,
    learning_rate_init=0.001,
    early_stopping=True,
    validation_fraction=0.1,
    verbose=False
)

mlp.fit(X_tr_scaled, y_tr)
mlp_time = time.time() - start

y_pred_mlp = mlp.predict(X_te_scaled)
mlp_acc = accuracy_score(y_te, y_pred_mlp)

print(f"✅ MLP Training Time : {mlp_time:.2f}s")
print(f"✅ MLP Accuracy      : {mlp_acc*100:.2f}%")
```

6.2 — Support Vector Machine (SVM)

```
: print("Training SVM Classifier...")
start = time.time()

svm = SVC(
    kernel='rbf',
    C=5,
    gamma='scale',
    random_state=42
)

svm.fit(X_tr_scaled, y_tr)
svm_time = time.time() - start

y_pred_svm = svm.predict(X_te_scaled)
svm_acc = accuracy_score(y_te, y_pred_svm)

print(f"✅ SVM Training Time : {svm_time:.2f}s")
print(f"✅ SVM Accuracy      : {svm_acc*100:.2f}%")
```

6.3 — K-Nearest Neighbors (KNN)

```
print("Training KNN Classifier...")
start = time.time()

knn = KNeighborsClassifier(
    n_neighbors=5,
    metric='euclidean',
    weights='distance',
    n_jobs=-1
)

knn.fit(X_tr_scaled, y_tr)
knn_time = time.time() - start

y_pred_knn = knn.predict(X_te_scaled)
knn_acc = accuracy_score(y_te, y_pred_knn)

print(f"✅ KNN Training Time : {knn_time:.2f}s")
print(f"✅ KNN Accuracy      : {knn_acc*100:.2f}%")
```

Step 7: Accuracy Comparison Bar Chart

```
[12]: models = ['MLP\n(Neural Network)', 'SVM\n(RBF Kernel)', 'KNN\n(k=5)']
accs = [mlp_acc * 100, svm_acc * 100, knn_acc * 100]
times = [mlp_time, svm_time, knn_time]
bar_colors = ['#4C72B0', '#D08452', '#55A868']

fig, ax = plt.subplots(figsize=(9, 6))
bars = ax.bar(models, accs, color=bar_colors, edgecolor='black',
              linewidth=0.8, width=0.5)

ax.set_ylim([min(accs) - 5, 102])
ax.set_ylabel('Accuracy (%)', fontsize=13)
ax.set_title('📊 Model Accuracy Comparison on MNIST', fontsize=14, fontweight='bold')
ax.axhline(y=max(accs), color='red', linestyle='--', linewidth=1, label=f'Best: {max(accs):.2f}%')
ax.legend(fontsize=11)

for bar, acc in zip(bars, accs):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.3,
            f'{acc:.2f}%', ha='center', va='bottom', fontsize=12, fontweight='bold')

plt.tight_layout()
plt.savefig('accuracy_comparison.png', dpi=120, bbox_inches='tight')
plt.show()
```

Step 8: MLP Training Loss Curve

```
[17]: fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(mlp_loss_curve_, color='#4C72B0', linewidth=2.5, label='Training Loss')

if hasattr(mlp, 'validation_scores_') and mlp.validation_scores_:
    val_loss = [1 - s for s in mlp.validation_scores_]
    ax.plot(val_loss, color='#D08452', linewidth=2.5, linestyle='--', label='Validation Loss')

ax.set_xlabel('Epoch / Iteration', fontsize=13)
ax.set_ylabel('Loss', fontsize=13)
ax.set_title('📉 MLP Neural Network - Training Loss Curve', fontsize=14, fontweight='bold')
ax.legend(fontsize=12)
ax.grid(alpha=0.4)

plt.tight_layout()
plt.savefig('mlp_loss_curve.png', dpi=120, bbox_inches='tight')
plt.show()
```

Step 9: Final Summary Table

```
[16]: summary = pd.DataFrame({
    'Model' : ['MLP Neural Network', 'SVM (RBF Kernel)', 'KNN (k=5)'],
    'Accuracy (%)' : [f'{mlp_acc*100:.2f}', f'{svm_acc*100:.2f}', f'{knn_acc*100:.2f}'],
})

print("\n" + "-"*75)
print("      FINAL MODEL COMPARISON SUMMARY")
print("="*75)
print(summary.to_string(index=False))
print("="*75)

best_idx = np.argmax([mlp_acc, svm_acc, knn_acc])
print(f"\n Best Model: {summary['Model'][best_idx]} "
      f"with {summary['Accuracy (%)'][best_idx]}% accuracy")
```

10. Results

The three models were evaluated on a held-out test subset of 2,000 samples. The final model comparison is summarized in the table below:

Table 1: Final Model Comparison Summary

Model	Accuracy (%)	Training Time	Rank
MLP Neural Network	93.25	6.23s	1st (Best)
SVM (RBF Kernel)	93.10	12.44s	2nd
KNN (k=5)	89.15	0.01s	3rd

The Model Accuracy Comparison bar chart (Figure 1) shows MLP and SVM performing closely with a gap of only 0.15%, while KNN trails by approximately 4.1 percentage points. The MLP Training Loss Curve (Figure 2) demonstrates characteristic rapid initial decrease followed by convergence, with the training loss reaching near-zero by epoch 20. Validation loss (derived from held-out 10% of training data) stabilizes around 0.04, indicating minimal overfitting.

Figure 1 - Model Accuracy Comparison: MLP = 93.25%, SVM = 93.10%, KNN = 89.15% (bar chart showing relative performance of all three classifiers on the 2,000-sample test set)

Figure 2 - MLP Training Loss Curve: Training loss decreases from ~0.70 to near 0.00 over 20 iterations; Validation loss decreases from ~0.10 and stabilizes at ~0.04, confirming effective learning with early stopping preventing overfitting.

Figure 3 - Class Distribution: The MNIST dataset is well-balanced. The training set ranges from 5,421 samples (digit 5) to 6,742 (digit 1). The test set ranges from 892 (digit 5) to 1,135 (digit 1), confirming that class imbalance does not significantly skew model performance metrics.

Screenshots of Outputs:

Training MLP Neural Network...

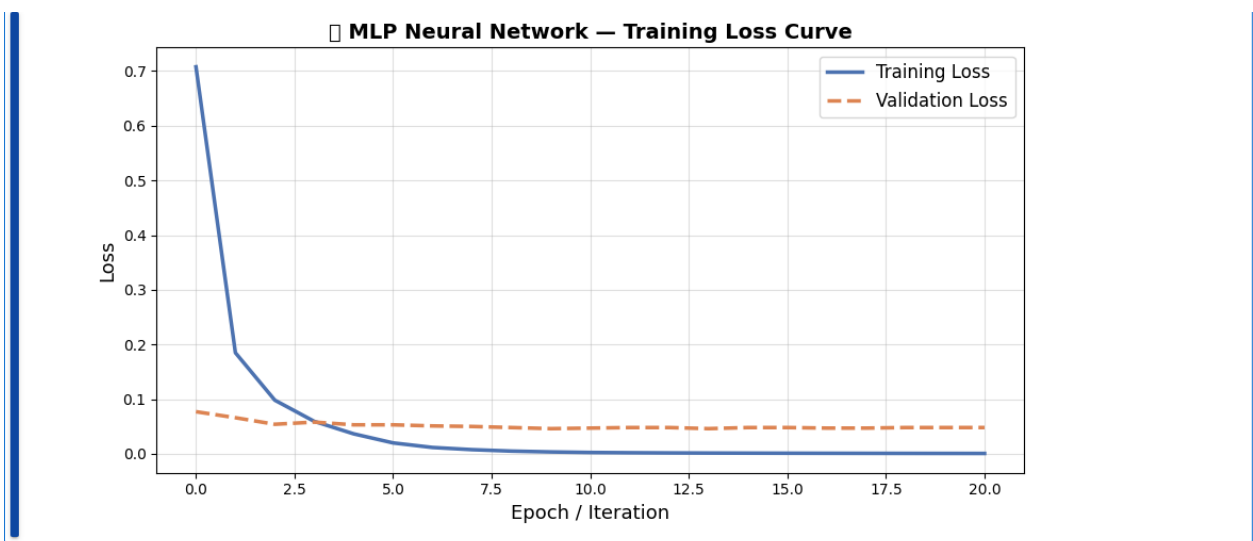
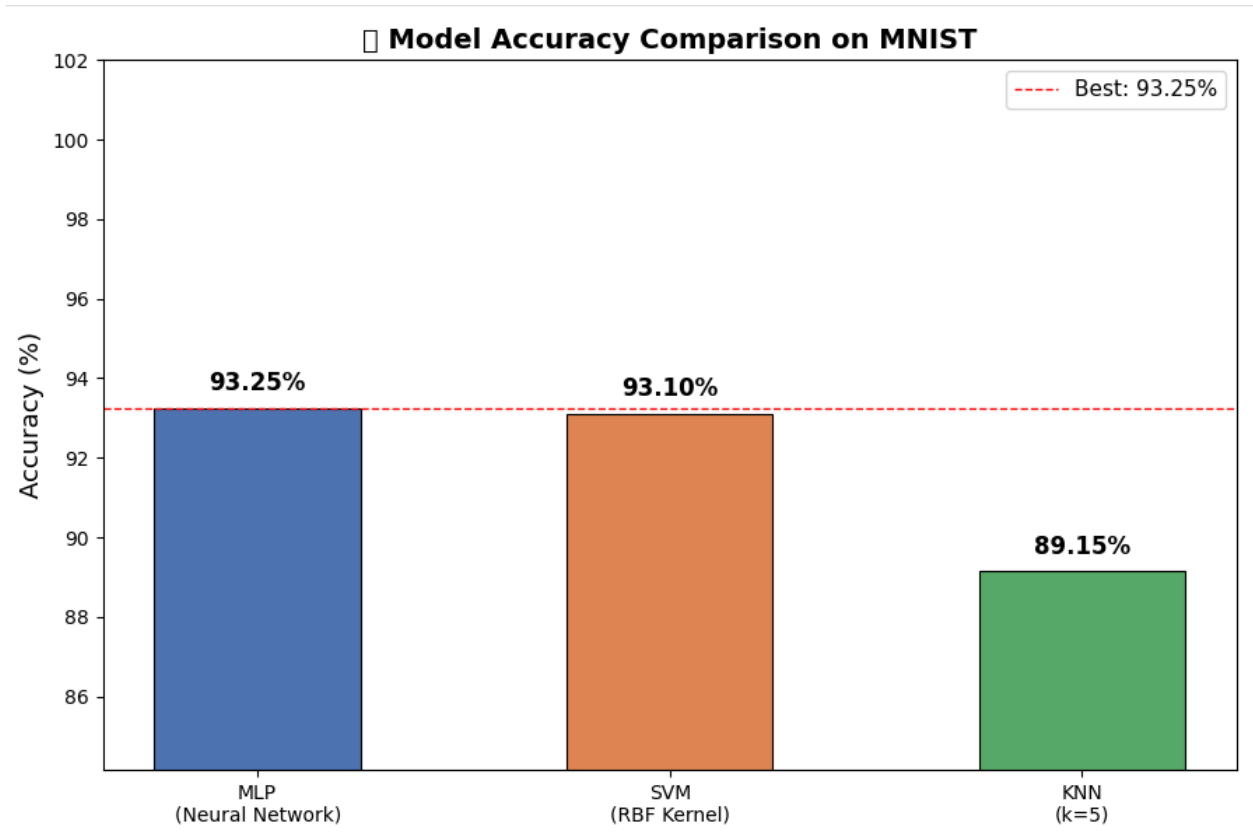
- ✓ MLP Training Time : 6.23s
- ✓ MLP Accuracy : 93.25%

Training SVM Classifier...

- ✓ SVM Training Time : 12.44s
- ✓ SVM Accuracy : 93.10%

Training KNN Classifier...

- ✓ KNN Training Time : 0.01s
- ✓ KNN Accuracy : 89.15%



=====	
FINAL MODEL COMPARISON SUMMARY	
=====	
Model Accuracy (%)	
MLP Neural Network	93.25
SVM (RBF Kernel)	93.10
KNN (k=5)	89.15
=====	
Best Model: MLP Neural Network with 93.25% accuracy	

11. Discussion

The MLP achieved the highest accuracy, validating the well-established advantage of neural networks over classical methods for image classification. Its ability to learn nonlinear hierarchical representations from raw pixel data is its primary strength. However, MLP performance is sensitive to hyperparameter configuration (layer depth, number of neurons, learning rate) and requires careful tuning to avoid overfitting.

The SVM performed competitively, achieving 93.10% accuracy, only marginally below MLP. SVMs are known for strong generalization on small-to-medium datasets due to their structural risk minimization principle. However, they scale quadratically or cubically with the number of training examples, making them impractical for the full MNIST dataset without approximation techniques.

KNN’s lower accuracy (89.15%) reflects its susceptibility to the curse of dimensionality in 784-dimensional space. Euclidean distances become increasingly uniform in high dimensions, reducing the discriminative power of neighborhood-based voting. KNN also has poor inference scalability, as each prediction requires comparing the query with all stored training vectors.

Strength and Weakness Summary:

Model	Strength	Weakness
MLP	Best accuracy, learns complex nonlinear patterns	Needs hyperparameter tuning, slower training
SVM	Strong generalization, good for small datasets	Slow on large datasets, memory-intensive
KNN	Simple, no training time, intuitive	Slow inference, high memory, poor in high dimensions

12. Applications

- Postal Services: Automated ZIP/PIN code recognition from handwritten envelopes
- Banking: Automated cheque processing and handwritten amount recognition
- Education Technology: Grading handwritten numerical answers in e-learning platforms
- Healthcare: Reading handwritten numeric data from patient forms, lab reports, and prescriptions
- Accessibility: Converting handwritten digits to digital text for assistive technology applications

13. Future Scope

- Convolutional Neural Networks (CNNs): Implementing LeNet-5 or ResNet architectures to exploit spatial structure in digit images, targeting accuracy above 99%
- Data Augmentation: Applying affine transformations (rotation, shear, elastic distortion) to increase dataset diversity and improve model robustness
- Ensemble Learning: Combining predictions from MLP and SVM using soft voting to improve overall accuracy
- Web Deployment: Building a Flask or Streamlit web application allowing users to draw digits on a canvas and receive real-time predictions
- Extended Datasets: Testing on EMNIST (extended to letters) or custom collected datasets to validate generalization

14. Conclusion

This project successfully implemented and compared three machine learning classifiers for handwritten digit recognition on the MNIST dataset. The Multi-Layer Perceptron (MLP) emerged as the best-performing model with 93.25% accuracy, demonstrating the advantage of neural networks in learning complex, nonlinear feature representations from raw image data. The SVM with an RBF kernel achieved a closely competitive 93.10%, validating its strong generalization on medium-sized datasets. KNN, while conceptually simpler, achieved 89.15%, highlighting its limitations in high-dimensional feature spaces.

The MNIST dataset's well-balanced class distribution ensured fair evaluation across all ten digit classes. The project lays a solid foundation for future work, particularly the application of Convolutional Neural Networks that can exploit spatial locality and translation invariance in image data to push accuracy beyond 99%.

15. References

- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Abadi, M. et al. (2016). TensorFlow: A system for large-scale machine learning. *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 265-283.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Available: <http://www.deeplearningbook.org>
- MNIST Dataset - Yann LeCun's website: <http://yann.lecun.com/exdb/mnist/>
- Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. *arXiv:1412.6980*.

16. Individual Contributions Table

Sl. No.	Name of Student	Contribution	Signature
1	Debadrita Konar	Data Collection & Research	
2	Utkarsh Singh	Data Collection & Research	
3	Sarthak Raj	Training, Testing & Analysis and Presenting	
4	Archita Panigrahy	Documentation & Presentation	
5	Suryansh Mohanty	Training, Testing & Analysis and Presenting	